Chapter 2: Getting Started

In this course, we will introduce some primary topics in algorithms, covering a number of interesting problems, and we will learn some useful techniques for designing efficient algorithms. In addition, we will learn how to prove the correctness of these algorithms and analyze their *running time*.[1]

1 Algorithms for solving a problem

An *algorithm* is a well-defined procedure consisting of a sequence of computational steps that is capable of solving a given problem. The algorithm takes some specific (given) values from the problem statement as *input* and produces the results of wanted as *output*. See, for example, the sorting problem and the algorithm below.

1.1 Sorting with the Insertion-Sort

Consider the sorting problem: given a sequence of n numbers in an array $A = \langle a_1, a_2, \ldots, a_n \rangle$, find a permutation $A = \langle a'_1, a'_2, \ldots, a'_n \rangle$ such that $a'_1 \leq a'_2 \leq \ldots \leq a'_n$.

This problem can be solved by the *insertion-sort* (in *pseudocode*) given on Page 18. The insertion-sort solves the problem using an *incremental* approach: assume that we have sorted the subarray $A[1 \ldots j-1]$; we then insert the single element $A[j]$ into a proper place, yielding the sorted subarray $A[1 \ldots j]$. Fig. 2.2 shows how the algorithm works for $A = \langle 5, 2, 4, 6, 1, 3 \rangle$.

Note that the algorithm sorts the numbers *in place*: that is, it rearranges the numbers within the array A, with at most a constant number of them stored outside the array at any time. It is easy to see that the insertion-sort is correct, but a question arises. Can we mathematically prove the correctness of the insertion sort? The answer is yes—by mathematical induction—as we will see below.

1.2 Proving the correctness of a loop in an algorithm

To prove the correctness of an algorithm involving *loops*, the notion of *loop invariants* is essential. By a loop invariant, we mean that it is a collection of *some properties* that construct the loop or help show the correctness of the loop. We have to verify that the loop invariant holds in Initialization, Maintenance, and Termination; i.e.,

- Initialization: It is true prior to the first iteration of the loop.
- Maintenance: If it is true before an iteration of a loop, it remains true before the next iteration.
- Termination: When the loop terminates, the invariant holds.

[1] Throughout the course, we use the book *Introduction to Algorithms* (third edition) by Cormen, Leiserson, Rivest, and Stein.

The loop invariant then implies the correctness of the loop. The correctness of the algorithm then follows from the correctness of the loops.

For example, the insertion-sort contains a "for" loop, for which the loop invariant is the property (according to the book): "At the start of each iteration of the *for* loop of lines 1–8, the subarray $A[1 \dots j-1]$ consists of the elements originally in $A[1 \dots j-1]$, but in sorted order". By verifying that the loop invariant holds in the Initialization, Maintenance, and Termination, we prove the correctness of the "for" loop and thus the correctness of the algorithm.

This way of proving a loop (and/or an algorithm) is similar to the mathematical induction of proving a mathematical result, where we first prove a base case and then an inductive step. Conceptually, the Initialization corresponds to the base case, and the Maintenance corresponds to the inductive step. The Termination is nothing but stopping the "induction".

We remark that a loop invariant can be actually anything that helps prove the loop (and/or the algorithm): it can be an equation and/or any statement of your preference. For example, for the insertion-sort, the statement is simpler: between line 1 and line 2, $A[1 \dots j-1]$ is a sorted array.

2 Analyzing algorithms

Besides the correctness of an algorithm, another aspect of interest is the computational time of an algorithm, i.e., how much time the algorithm needs to solve the problem, which is commonly termed as *running time*. Other aspects such as the required memory, communication bandwidth, and computer hardware are also of interest, but, in this course, we focus on running time, and we learn how to analyze it (below and later on).

2.1 The random-access machine (RAM) model

To analyze the running time of an algorithm, we need a proper model such that the running time can be meaningful and independent of any languages and machines used to implement the algorithm. In addition, under the model, the running time of different algorithms for solving a problem can be fairly compared. The *random-access machine (RAM)* model for algorithms is such a model, which assumes that

- Data types are integer and floating point (for storing real numbers).
- Algorithms use common instructions available in real computers: e.g., arithmetic (add, subtract, multiply, divide, remainder, floor, ceiling), data movement (load, store, copy), and control (conditional and unconditional branch).
- Each such instruction takes a constant amount of time.
- Instructions are executed one after another, with no concurrent operations.

Normally, the time taken by an algorithm grows with the size of the input; it is, therefore, customary to describe the running time of an algorithm in terms of *the size of its input*. For insertion sort, the *input size* is the number of items in the input array. (For other problems, the input size that best quantifies the running time may be different.)

To conclude, with the RAM model, we analyze the running time of an algorithm by

1. Assuming that each primitive operation (or "basic steps") takes a constant amount of time, and then
2. Counting the total number of primitive operations or "basic steps" that are executed throughout the algorithm.

In the end, we will obtain the running time expressed as a function of the input size. Below, we give a detailed analysis of the running time of the insertion sort.

2.2 Running time of Insertion-Sort

We analyze the insertion-sort by assuming that each execution of the i-th line takes a constant time c_i and counting the number of executions for each line. Let t_j denote the number of executions of line 5 executed in the j-th iteration of the *for loop* (since the number of executions of the *while loop* (lines 5–7) may be different in different *for loop* iterations). Then the running time of the insertion-sort on an input array of size n can be obtained as follows: we refer to the insertion-sort given on Page 26 and obtain

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2}^{n} t_j + (c_6 + c_7) \sum_{j=2}^{n} (t_j - 1) + c_8(n-1).$$

In the special case where the input array $A[1 \ldots n]$ is sorted—the best case—we have $t_j = 1$ and thus

$$\begin{aligned} T(n) &= c_1 n + (c_2 + c_4)(n-1) + c_5(n-1) + c_8(n-1) \\ &= (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8) \\ &= an + b \end{aligned}$$

with $a \triangleq c_1 + c_2 + c_4 + c_5 + c_8$ and $b \triangleq c_2 + c_4 + c_5 + c_8$; in this case, $T(n)$ is a *linear function* of n. If $A[1 \ldots n]$ is in reverse sorted order—the worst case—we have $t_j = j$ and thus

$$\begin{aligned} T(n) &= c_1 n + (c_2 + c_4)(n-1) + c_5 \left(\frac{n(n+1)}{2} - 1 \right) + (c_6 + c_7) \left(\frac{n(n-1)}{2} \right) + c_8(n-1) \\ &= \left(\frac{c_5 + c_6 + c_7}{2} \right) n^2 + \left(c_1 + c_2 + c_4 + \frac{c_5 - c_6 - c_7}{2} + c_8 \right) n - (c_2 + c_4 + c_5 + c_8). \end{aligned}$$

We can express this worst-case running time as $an^2 + bn + c$ with a, b, c denoting the respective coefficients in $T(n)$; in this case, the running time is a *quadratic function* of n.

The *worst-case running time* is usually the main concern when we evaluate an algorithm, because it provides a bound (or guarantee) telling us that the situation cannot be worse than that, and also it occurs fairly often in many problems (or algorithms). Moreover, we usually find that the *average case* (or typical case) is roughly as bad as the worst case.

For the insertion-sort, we may think of the *average case* by randomly choosing n integers and applying the insertion-sort, in which we have roughly half of integers in $A[1 \ldots j-1]$ smaller than $A[j]$ and thus $t_j = \frac{j}{2}$. The resulting average-case running time turns out to be also a quadratic function of the input size, just like the worst-case running time.

From the above analysis, we see that in any case, the running time $T(n)$ of the insertion-sort is a polynomial of the input size n. Algorithms of this kind are termed the *polynomial-time* algorithms.

2.3 Order of growth (rate of growth)

For insertion-sort, the running time of the best case is $an + b$ for some a and b, the average case is $an^2 + bn + c$ for some a, b, c, and the worst case is $an^2 + bn + c$ for some other a, b, c. Note that $a \neq 0$ in these three cases. Using the description of *order of growth*, the running time of the three cases can be further simplified to $T(n) = \Theta(n)$ for the best case, and $T(n) = \Theta(n^2)$ for both the average case and the worst case. In this simplified description, we consider only the *degree* of the polynomial, which most concisely expresses the efficiency of an algorithm.

The idea of this description stems from that the low-order terms of a polynomial in n are relatively less important than the leading term, and thus they can be ignored when n is large. By further ignoring the coefficient of the leading term, i.e., considering only the degree, we obtain the description of running time in Θ. A formal definition of Θ, along with some other notations related to Θ, will be introduced in Chapter 3. The notion of order of growth is the main topic of Chapter 3.

3 Designing Algorithms

In this section, we introduce a powerful approach to algorithm design, called divide-and-conquer. We exemplify this approach with the merge-sort, which solves the sorting problem in a different "recursive" manner. We will see that its worst-case running time is less than that of insertion sort.

3.1 The divide-and-conquer approach

Divide-and-conquer is a "recursive-in-level" approach to designing an algorithm. This approach consists of the following three steps at each level of the recursion:

1. Divide the problem into several subproblems that are the same as (or similar to) the original problem but smaller in size.

2. Conquer the subproblems by solving them recursively. However, if the subproblems' sizes are small enough, just solve the subproblems in a straightforward manner.

3. Combine the solutions to the subproblems into the solution for the original problem.

Step 1 is conceptually simple; Step 2 involves only a functional call to itself; Step 3 is *usually* the key part, requiring an efficient way to combine the solutions of the subproblems.

The algorithm Merge-Sort(A, p, r) below is designed with the divide-and-conquer method, where A is an array; p and r are indices into the array with $p \leq r$.

Merge-Sort(A, p, r)

```
1    if p < r
2        q = ⌊(p + r)/2⌋
3        Merge-Sort(A, p, q)
4        Merge-Sort(A, q + 1, r)
5        Merge(A, p, q, r)
```

The algorithm works as follows. If $p < r$, we divide the array $A[p \dots r]$ into two subarrays and sort the two subarrays (i.e., solves the two subproblems). In Line 5, we merge the solutions of the two subproblems. The procedure Merge(A, p, q, r) is given in Page 31, which takes time $\Theta(n)$, $n \triangleq r - p$, to merge the two sorted subarrays $A[p \dots q]$ and $A[p+1 \dots r]$ into a new sorted $A[p \dots r]$. For details, we refer to Pages 31–34.

3.2 Proving the correctness of a divide-and-conquer algorithm

To prove the correctness of an algorithm (procedure) designed with the divide-and-conquer, we use mathematical induction, which consists of the three steps:

- Base case: prove that the algorithm (the procedure) works for small examples.
- Inductive hypothesis: assume that the solution is correct for all sub-problems.
- Induction step: show that if the inductive hypothesis is correct, then the algorithm (the procedure) is correct for the original problem.

Now we use the mathematic induction to prove the Merge-Sort(A, p, r): we first assume that the merge procedure in Merge-Sort(A, p, r) is correct; we then translate Merge-Sort(A, p, r) into the following procedure and prove its correctness

Merge-Sort $A[1 \dots, n]$

```
1   if n ≤ 1, done; otherwise,
2   recursively sort A[1 ... ⌈n/2⌉] and A[⌈n/2⌉ + 1 ... n]
3   merge the two sorted arrays
```

Proof:

1. Base case: if $n = 1$, the algorithm will return the correct answer because $A[1 \dots 1]$ is already sorted; if $n = 0$; no work to do.
2. Inductive hypothesis: assume that $A\big[1 \dots \lceil \frac{n}{2} \rceil\big]$ and $A\big[\lceil \frac{n}{2} \rceil + 1 \dots n\big]$ are correctly sorted.
3. Induction step: If both $A\big[1 \dots \lceil \frac{n}{2} \rceil\big]$ and $A\big[\lceil \frac{n}{2} \rceil + 1 \dots n\big]$ are correctly sorted, then under the assumption that the merge procedure is correct, the whole array $A[1 \dots, n]$ is sorted after merging.

3.3 Analyzing divide-and-conquer algorithms

When an algorithm contains recursive calls to itself, we can usually describe its running time with a *recurrence equation*, which describes the overall running time on a problem of input size n in terms of the running time on the subproblems of smaller inputs.

For a divide-and-conquer algorithm, the *recurrence* can be obtained as follows. First, we let $T(n)$ denote the running time on a problem of size n. If n is small enough, say $n \le c$ for some constant c, the straightforward solution takes constant time, which can be expressed by $\Theta(1)$. Suppose that we divide the problem into a subproblems, each of which is $1/b$ the size of the original. We will then need $T(n/b)$ to solve a subproblem and in total $aT(n/b)$ to solve a of them. We therefore obtain the following recurrence:

$$T(n) = \begin{cases} \Theta(1) & \text{if } n \leq c, \\ aT(n/b) + D(n) + C(n) & \text{otherwise,} \end{cases}$$

where $D(n)$ denotes the time taken to divide the problem (into the a subproblems) and $C(n)$ denotes the time to combine the solutions to the subproblems into the solution to the original problem. In Chapter 4, we will learn a systematic method to solve recurrence equations of this form.

3.3.1 Analysis of merge-sort

For merge-sort, we have $a = b = 2$, $D(n) = \Theta(1)$, and $C(n) = \Theta(n)$. The worst-case running time of merge-sort can then be expressed as

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1, \\ 2T(n/2) + \Theta(1) + \Theta(n) & \text{otherwise,} \end{cases} \tag{1}$$

$$= \begin{cases} \Theta(1) & \text{if } n = 1, \\ 2T(n/2) + \Theta(n) & \text{otherwise,} \end{cases} \tag{2}$$

where the last equation follows from that $\Theta(1)$ and $\Theta(n)$ adds to $\Theta(n)$. By the "Master Theorem" in Chapter 4, we obtain $T(n) = \Theta(n \log_2 n)$.

For this simple recurrence (2), we can easily obtain the result $T(n) = \Theta(n \log_2 n)$ without referring to the Master theorem. Let's see how we can make it. First, we rewrite (2) as

$$T(n) = \begin{cases} c_1 & \text{if } n = 1, \\ 2T(n/2) + c_2 n & \text{otherwise,} \end{cases} \tag{3}$$

where the constant c_1 represents the time required to solve problems of size 1 and where c_2 represents the constant originally ignored in $\Theta(n)$. Then, let $c \triangleq \max\{c_1, c_2\}$. We can further rewrite $T(n)$ as

$$T(n) = \begin{cases} c & \text{if } n = 1, \\ 2T(n/2) + c \cdot n & \text{otherwise.} \end{cases} \tag{4}$$

By expanding out the recurrence (4) as in Figure 2.5 in Page 38, we obtain

$$T(n) = cn \log_2 n + cn \tag{5}$$

$$= \Theta(cn \log_2 n + cn) \tag{6}$$

$$= \Theta(n \log_2 n), \tag{7}$$

where the step to $\Theta(cn \log_2 n + cn)$ is an "equivalence relation" which will be formally defined in Chapter 3.